COMP0034 Coursework 2 Report

1. Test Critique

Building and testing this full-stack application involved four distinct testing strategies, each with different trade-offs in speed, realism, and maintainability.

Unit tests with pytest provided the fastest feedback. For Coursework 1's data access layer, 35 tests directly called functions like `get_categories()` and completed in under a second with no browser or network involvement. Each function was validated in isolation, making it straightforward to locate problems. The weakness is that these tests only validate individual layers independently: they cannot confirm whether the frontend correctly consumes data, or whether user interactions trigger the expected backend behaviour.

FastAPI TestClient was used for Coursework 2 to test API routes, yielding 24 tests covering GET, POST, PATCH, and DELETE endpoints. It sends real HTTP requests to the app, exercising routing, validation, serialisation, and error handling — catching bugs like incorrect status codes — while using an in-memory SQLite database to ensure test isolation. Tests ran quickly (24 tests in ~0.3 seconds) because there was no browser overhead. However, `TestClient` does not exercise the frontend at all. True full-stack integration would require starting both services simultaneously and using Selenium to interact with the UI while monitoring API calls — a gap I did not address due to time constraints.

Web-browser tests with Selenium filled the UI validation gap. The 22 tests in `test_browser.py` launched Chrome, navigated the dashboard, and verified that charts rendered SVGs, KPI cards displayed numbers, and filters updated the page correctly. These caught issues unit tests would miss — a typo in a component ID breaks the UI but passes all backend tests. The downside was speed and fragility: browser tests took 30–60 seconds and occasionally failed due to timing issues with animations. Debugging produced screenshots rather than stack traces, which slowed diagnosis and meant I ran them less frequently during development.

pytest fixtures are a setup mechanism provided by pytest, defined using the `@pytest.fixture` decorator. They automatically prepare the required environment before tests run and clean up afterwards, avoiding repetitive initialisation code in each test. In Coursework 1, a `dash_app` fixture created a fresh Dash instance per test module. In Coursework 2, a `client` fixture set up an in-memory database, seeded it with sample data (3 categories, 4 markets), and yielded a `TestClient`, making tests more concise and readable. Using `app.dependency_overrides` to inject the test database session was elegant, though it is a FastAPI-specific pattern that would not transfer to other frameworks.

Test coverage reporting via `pytest-cov` highlighted blind spots. The API backend achieved 93.50% coverage in isolation, with `routes.py` at 89.55%. The 14 missed lines correspond to error-handling paths — catching `IntegrityError` on duplicate inserts or returning 404 for non-existent resources — that are difficult to trigger deterministically. The full test suite showed 80.46% overall coverage, with `callbacks.py` at only 26.36% because Dash callbacks only execute during browser interactions; the 22 Selenium tests covered that gap. Coverage numbers are not a perfect proxy for quality, but they guided me toward untested paths.

Figure 1: Full Test Suite Coverage Report (Coursework 1 + Coursework 2)

File ▲	statements	missing	excluded	coverage
src/tourism_dashboard/__init__.py	1	0	0	100.00%
src/tourism_dashboard/api/__init__.py	0	0	0	100.00%
src/tourism_dashboard/api/database.py	11	2	0	81.82%
src/tourism_dashboard/api/main.py	21	2	0	90.48%
src/tourism_dashboard/api/models.py	38	0	0	100.00%
src/tourism_dashboard/api/routes.py	134	14	0	89.55%
src/tourism_dashboard/api/schemas.py	73	0	0	100.00%
src/tourism_dashboard/app.py	13	1	0	92.31%
src/tourism_dashboard/callbacks.py	110	81	0	26.36%
src/tourism_dashboard/data_access.py	96	2	0	97.92%
src/tourism_dashboard/layout.py	25	0	0	100.00%
Total	**522**	**102**	**0**	**80.46%**

Figure 2: API Backend Coverage Report (test_api_routes.py only)

File ▲	statements	missing	excluded	coverage
src/tourism_dashboard/api/__init__.py	0	0	0	100.00%
src/tourism_dashboard/api/database.py	11	2	0	81.82%
src/tourism_dashboard/api/main.py	21	2	0	90.48%
src/tourism_dashboard/api/models.py	38	0	0	100.00%
src/tourism_dashboard/api/routes.py	134	14	0	89.55%
src/tourism_dashboard/api/schemas.py	73	0	0	100.00%
Total	**277**	**18**	**0**	**93.50%**

Arrange-Act-Assert structure kept tests consistent and readable. For example, `test_create_market_success` arranges a market dictionary, acts by posting to `/api/v1/markets`, and asserts a 201 response. Complex setups involving multiple related records could make the Arrange section verbose, but the pattern remained useful for scanning and debugging tests quickly.

---

2. References, AI Acknowledgement and Dataset Attribution

2.1 References

Key documentation sources:

- FastAPI Documentation: https://fastapi.tiangolo.com/
- SQLModel Documentation: https://sqlmodel.tiangolo.com/
- Pydantic Documentation: https://docs.pydantic.dev/
- httpx Documentation: https://www.python-httpx.org/

2.2 Acknowledgement of AI Use

No Generative AI tools were used in the development of the application code, test suites, or the writing of this report.

2.3 Dataset Attribution

Source: International Visitor Arrivals By Inbound Tourism Markets (Sea), Monthly. Used under the **Open Government Licence v3** .